ENSIAS

N+ONE DATACENTERS

École Nationale Supérieure d'Informatique et d'Analyse des Systèmes – RABAT

Rapport de Stage de 2ème Année

Filière Génie Logiciel

Développement d'une Application Web Sécurisée pour la Gestion des Appels d'Offres

---

Réalisé au sein de : **N+ONE DATACENTERS, Casablanca**

Période : **06 août – 06 septembre 2025**

Réalisé par :

Anas Madani

Encadré par :

M. Ayoub Amellaln (N+ONE)
M. Amine El Mahi (N+ONE)

Année universitaire 2024/2025

Dédicace

À mes parents, pour leur soutien indéfectible, leurs sacrifices et leur confiance tout au long de ce parcours. Que Dieu vous accorde santé, bonheur et longue vie.

À ma famille et mes amis, pour votre bienveillance et vos encouragements qui ont rendu ce travail plus léger.

- Anas



Remerciements

Je tiens tout d'abord à remercier mes encadrants au sein de ***N+ONE DATACENTERS****,* ***M. Ayoub Amellaln*** *et* ***M. Amine El Mahi****, pour leur confiance, leur disponibilité et leurs conseils tout au long de ce stage. Merci de m'avoir guidé avec bienveillance et de m'avoir offert un cadre de travail stimulant.*

*Mes remerciements vont également au corps enseignant de l'****ENSIAS*** *pour la formation solide dispensée tout au long de ces deux années, et en particulier aux enseignants de la filière Génie Logiciel.*

Enfin, merci à toute l'équipe de N+ONE DATACENTERS pour l'accueil chaleureux et pour avoir facilité mon intégration dès le premier jour.

Résumé

Ce rapport présente le développement d'une application web sécurisée de gestion des appels d'offres, réalisé lors d'un stage d'un mois au sein de **N+ONE DATACENTERS** à Casablanca. Le projet répond à un besoin concret : remplacer un processus fragmenté (emails, Excel, partages réseau) par une solution centralisée, traçable et sécurisée.

L'application repose sur une architecture monolithe modulaire (**Django REST Framework** côté backend, **React.js** côté frontend) déployée via **Docker Compose** avec un proxy Nginx en HTTPS. Elle couvre l'intégralité du cycle de vie d'un appel d'offres : de la création au brouillon jusqu'à l'attribution du marché, en passant par la publication, la collecte des soumissions et l'évaluation des candidats.

Les principales fonctionnalités de sécurité implémentées sont : l'authentification **JWT** avec access token en mémoire et refresh token en cookie `httpOnly` scopé, le contrôle d'accès basé sur les rôles (**RBAC**) à quatre niveaux (ADMIN, MANAGER, ANALYST, VIEWER), le chiffrement sélectif **AES-256-GCM** des documents sensibles, et un journal d'audit complet. Un portail public isolé permet aux sociétés externes de déposer leurs propositions sans compte utilisateur, avec rate limiting et vérification des deadlines côté serveur.

Un pipeline d'intégration continue (GitHub Actions) exécute automatiquement les tests pytest et le lint flake8 à chaque push. L'application est déployable en une commande (`docker-compose up`) et inclut un script de démonstration peuplant la base avec un scénario complet.

Mots-clés : Appels d'offres, Django REST Framework, React.js, JWT, RBAC, AES-256-GCM, Docker Compose, GitHub Actions, Portail public, Audit.

Abstract

This report presents the development of a secure web application for procurement management, carried out during a one-month internship at **N+ONE DATACENTERS** in Casablanca. The project addresses a concrete need : replacing a fragmented process (emails, Excel, shared drives) with a centralized, traceable, and secure solution.

The application is built on a modular monolith architecture (**Django REST Framework** backend, **React.js** frontend) deployed via **Docker Compose** with an Nginx HTTPS proxy. It covers the complete procurement lifecycle : from draft creation to contract award, through publication, submission collection, and candidate evaluation.

The main security features implemented are : **JWT authentication** with access token in memory and refresh token in a scoped `httpOnly` cookie, four-level **role-based access control** (RBAC), selective **AES-256-GCM encryption** of sensitive documents, and a comprehensive audit log. An isolated public portal allows external companies to submit proposals without an account, with IP-based rate limiting and server-side deadline enforcement.

A continuous integration pipeline (GitHub Actions) automatically runs pytest tests and flake8 lint on every push. The application deploys in a single command (`docker-compose up`) and includes a demo seed script.

Keywords : Procurement management, Django REST Framework, React.js, JWT, RBAC, AES-256-GCM, Docker Compose, GitHub Actions, Public portal, Audit log.

Liste des abréviations

AES	Advanced Encryption Standard
API	Application Programming Interface
CI	Continuous Integration
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
DRF	Django REST Framework
GCM	Galois/Counter Mode
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JWT	JSON Web Token
MIME	Multipurpose Internet Mail Extensions
ORM	Object-Relational Mapper
PDF	Portable Document Format
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPA	Single Page Application
SQL	Structured Query Language
TLS	Transport Layer Security
UUID	Universally Unique Identifier
XSS	Cross-Site Scripting

Table des matières

Liste des abréviations **4**

Liste des figures **7**

Liste des tableaux **8**

Introduction Générale **1**

1 Cahier des Charges **3**
1.1 Présentation de l'organisme d'accueil . . . 3
1.1.1 N+ONE DATACENTERS . . . 3
1.1.2 Environnement de travail . . . 3
1.2 Présentation du sujet . . . 3
1.2.1 Les appels d'offres . . . 3
1.2.2 Les acteurs du système . . . 3
1.2.2.0.1 Acteurs internes . . . 4
1.2.2.0.2 Acteurs externes . . . 4
1.3 Analyse des besoins . . . 4
1.3.1 Besoins fonctionnels . . . 4
1.3.2 Besoins non-fonctionnels . . . 4
1.4 Évaluation et justification de la solution . . . 5
1.4.1 Étude de l'existant . . . 5
1.4.2 Critique de l'existant . . . 5
1.4.3 Justification du développement sur mesure . . . 6
1.4.3.0.1 Estimation des coûts d'infrastructure . . . 6

2 Analyse, Conception et Architecture **7**
2.1 Choix technologiques . . . 7
2.1.0.0.1 Django . . . 7
2.1.0.0.2 PostgreSQL . . . 7
2.1.0.0.3 Docker Compose . . . 7
2.2 Planification . . . 8
2.3 Modélisation des besoins . . . 8
2.3.1 Diagramme de cas d'utilisation . . . 8
2.3.2 Diagramme d'activité – Cycle de vie d'une offre . . . 8
2.4 Architecture de la solution . . . 8
2.4.1 Vue globale . . . 8
2.4.2 Monolithe modulaire vs microservices . . . 9
2.4.3 Services Docker Compose . . . 9
2.5 Conception détaillée . . . 9
2.5.1 Modèle de données . . . 9
2.5.2 Conception de l'API REST . . . 10
2.5.3 Architecture de sécurité . . . 10
2.5.3.1 Authentification JWT . . . 10
2.5.3.2 Contrôle d'accès basé sur les rôles (RBAC) . . . 11
2.5.3.3 Chiffrement AES-256-GCM . . . 11
2.5.3.4 Diagramme de séquence – Soumission portail public . . . 12

3 Réalisation et Mise en Œuvre **13**
3.1 Environnement de développement . . . 13
3.1.1 Outils utilisés . . . 13
3.1.2 Structure du projet . . . 13
3.1.3 Pipeline d'intégration continue . . . 14
3.2 Implémentation du backend . . . 14
3.2.1 Authentification et RBAC . . . 14
3.2.2 Gestion des offres et machine à états . . . 15
3.2.3 Documents et chiffrement AES-256-GCM . . . 15
3.2.4 Portail public . . . 15
3.2.5 Audit et notifications . . . 16
3.3 Implémentation du frontend . . . 16
3.3.1 Gestion de l'authentification . . . 16
3.3.2 Routing protégé par rôle . . . 16
3.3.3 Portail public . . . 16
3.4 Interfaces et fonctionnalités . . . 17
3.4.1 Tableau de bord . . . 17
3.4.2 Liste et filtrage des offres . . . 17
3.4.3 Détail d'une offre . . . 17
3.4.4 Formulaire de création d'offre . . . 17
3.4.5 Évaluation d'une soumission . . . 17
3.4.6 Portail public . . . 17
3.4.7 Journal d'audit . . . 17
3.4.8 Rapport PDF . . . 17
3.5 Tests et validation . . . 17
3.5.1 Tests automatisés . . . 17
3.5.2 Résultats CI . . . 17
3.5.3 Tests manuels . . . 18

Conclusion **19**
3.5.3.0.1 Isolation du portail public . . . 19
3.5.3.0.2 Chiffrement sélectif . . . 19
3.5.3.0.3 Pipeline CI . . . 19

4 docker-compose.yml **21**

5 Exemple d'entrée AuditLog **23**

6 Manuel utilisateur **24**

7 Résultats du pipeline CI **25**

Table des figures

Liste des tableaux

1.1 Besoins fonctionnels du système 4
1.2 Comparaison des solutions existantes 5

2.1 Comparaison et justification des choix technologiques 7
2.2 Services Docker Compose 9
2.3 Principaux endpoints de l'API 10
2.4 Matrice de contrôle d'accès 11

3.1 Outils de développement 13
3.2 Couverture des tests automatisés 17

Introduction Générale

Dans un contexte économique où la commande publique et les marchés d'entreprise représentent des volumes financiers considérables, la gestion rigoureuse des appels d'offres constitue un enjeu stratégique majeur. Pour les organisations qui publient et évaluent régulièrement ces appels, la qualité du processus détermine directement la compétitivité, la conformité réglementaire et la maîtrise des délais.

Contexte du stage

Ce stage de deuxième année a été effectué au sein de **N+ONE DATACENTERS**, société spécialisée dans l'hébergement et la gestion d'infrastructure datacenter, basée à Casablanca, au Maroc. D'une durée d'un mois, du 6 août au 6 septembre 2025, il s'inscrit dans le cadre de la formation en Génie Logiciel à l'École Nationale Supérieure d'Informatique et d'Analyse des Systèmes (ENSIAS) de Rabat.

Problématique

Avant ce projet, la gestion des appels d'offres chez N+ONE reposait sur des outils non spécialisés : échanges par email, suivi sous tableurs Excel et stockage de documents sur disques partagés. Cette approche fragmentée engendrait plusieurs problèmes opérationnels :

- **Absence de centralisation** : les informations étaient dispersées entre boîtes mail, fichiers Excel et répertoires réseau, rendant le suivi global impossible sans consolidation manuelle.
- **Risque de dépassement de délais** : aucun mécanisme d'alerte automatique ne prévenait les responsables à l'approche des dates limites de soumission ou d'évaluation.
- **Sécurité insuffisante** : les offres financières (données sensibles) circulaient par email ou étaient stockées en clair sur des partages réseau, sans chiffrement ni contrôle d'accès granulaire.
- **Traçabilité absente** : aucun historique ne permettait de savoir qui avait modifié quoi, ni quand, compromettant la conformité et la gouvernance.
- **Surcharge avant les échéances** : la réception de plusieurs soumissions simultanées en fin de période créait un goulot d'étranglement, les agents devant saisir manuellement les informations des candidats.

Objectifs du stage

L'objectif principal était de concevoir et développer une **application web sécurisée de gestion des appels d'offres**, couvrant l'intégralité du cycle de vie : de la création de l'offre jusqu'à l'attribution du marché, en passant par la publication, la collecte des soumissions et l'évaluation des candidats.

Les objectifs techniques et fonctionnels définis en début de stage étaient les suivants :

1. Authentification sécurisée avec gestion des rôles (RBAC) ;
2. Cycle de vie complet des appels d'offres avec transitions d'état contrôlées ;
3. Portail public permettant aux sociétés externes de déposer leurs offres sans compte utilisateur ;
4. Chiffrement AES-256-GCM des documents financiers sensibles ;
5. Notifications automatiques (délais, changements de statut) ;
6. Génération de rapports PDF ;
7. Journal d'audit traçant toutes les actions sur les ressources ;
8. Déploiement Dockerisé reproductible avec pipeline CI/CD.

Plan du rapport

Ce rapport s'organise en trois parties principales.

La **Partie 1** présente le cahier des charges : après une introduction à l'organisme d'accueil et au domaine des appels d'offres, elle détaille l'analyse des besoins fonctionnels et non-fonctionnels, l'étude des solutions existantes et la justification du développement sur mesure.

La **Partie 2** couvre l'analyse, la conception et l'architecture : choix technologiques argumentés, planification, modélisation UML (cas d'utilisation, diagrammes d'activité, séquences) et conception détaillée du système (modèle de données, API REST, architecture de sécurité).

La **Partie 3** décrit la réalisation et la mise en œuvre : implémentation des modules backend et frontend, présentation des interfaces, et résultats des tests de validation.

Chapitre 1

Cahier des Charges

1.1 Présentation de l'organisme d'accueil

1.1.1 N+ONE DATACENTERS

[1-2]

1.1.2 Environnement de travail

Le stage s'est déroulé au sein du département *[TODO : nom du département]*, sous la supervision de M. Ayoub Amellaln et M. Amine El Mahi.

1.2 Présentation du sujet

1.2.1 Les appels d'offres

Un appel d'offres est une procédure formalisée par laquelle une organisation (le *maître d'ouvrage*) invite des fournisseurs ou prestataires potentiels à soumettre des propositions commerciales et techniques pour la réalisation d'un marché. Cette procédure garantit la mise en concurrence, la transparence et la traçabilité des décisions d'achat.

Le cycle de vie d'un appel d'offres se décompose en plusieurs phases distinctes, illustrées dans la figure **??** :

1. **BROUILLON (DRAFT)** : l'offre est en cours de rédaction, non visible par les candidats.
2. **PUBLIÉE (PUBLISHED)** : l'offre est ouverte, les sociétés externes peuvent soumettre leurs propositions via le portail public.
3. **ÉVALUATION** : la date limite de soumission est passée ; les responsables évaluent et scorent les candidatures reçues.
4. **ATTRIBUÉE (AWARDED)** : le marché est attribué au candidat retenu.
5. **ANNULÉE (CANCELLED)** : l'offre est interrompue à tout moment avant attribution.

1.2.2 Les acteurs du système

Le système distingue deux catégories d'utilisateurs :

1.2.2.0.1 Acteurs internes Membres du personnel de N+ONE disposant d'un compte applicatif, répartis en quatre rôles :

- **ADMIN** : administrateur système, gère les comptes utilisateurs et accède au journal d'audit complet.
- **MANAGER** : responsable d'offres, crée et pilote les appels d'offres, attribue les marchés, génère les rapports PDF.
- **ANALYST** : évaluateur, score les soumissions et consulte les dossiers des candidats.
- **VIEWER** : lecteur seul, consulte les offres publiées sans pouvoir modifier quoi que ce soit.

1.2.2.0.2 Acteurs externes Sociétés soumissionnaires accédant uniquement au portail public sans créer de compte. Elles peuvent consulter les offres ouvertes et déposer leurs propositions directement en ligne.

1.3 Analyse des besoins

1.3.1 Besoins fonctionnels

TABLE 1.1 – Besoins fonctionnels du système

Module	Description
Authentification	Connexion sécurisée, déconnexion, rafraîchissement de session
Gestion des offres	Création, modification, publication, transition de statut, annulation
Candidats et soumissions	Enregistrement des candidats, dépôt de soumissions, scoring technique et financier
Documents	Upload multipart, téléchargement sécurisé, chiffrement sélectif
Portail public	Consultation des offres publiées et dépôt de soumissions sans authentification
Notifications	Alertes automatiques : délais approchants, changements de statut, attribution
Rapports	Export PDF d'une offre (synthèse, tableau des scores, candidat retenu)
Audit	Journal des actions sur toutes les ressources (qui, quoi, quand, depuis où)
Administration	Gestion des comptes utilisateurs (création, désactivation, changement de rôle)

1.3.2 Besoins non-fonctionnels

Sécurité Authentification par JWT (token en mémoire, cookie httpOnly), contrôle d'accès basé sur les rôles (RBAC), chiffrement AES-256-GCM des documents sensibles, audit complet, protection contre les injections SQL, HTTPS obligatoire.

Performance Réponses API inférieures à 500 ms pour les listes paginées. Pagination côté serveur (20 offres par page). Chiffrement uniquement sur les documents marqués sensibles pour ne pas pénaliser les uploads standards.

Disponibilité Redémarrage automatique des conteneurs Docker en cas d'échec (`restart: unless-stopped`). Vérifications de santé (*health checks*) sur chaque service.

Traçabilité Toute action d'écriture (création, modification, téléchargement, export, connexion) est enregistrée dans le journal d'audit avec l'adresse IP, l'utilisateur et le contenu des modifications.

Maintenabilité Architecture modulaire (8 applications Django indépendantes), pipeline CI automatique (tests + lint à chaque push), structure de projet documentée.

Reproductibilité Déploiement en une commande (`docker-compose up`). Variables d'environnement pour tous les secrets, aucune valeur en dur dans le code.

1.4 Évaluation et justification de la solution

1.4.1 Étude de l'existant

Plusieurs solutions existent sur le marché pour la gestion des appels d'offres :

TABLE 1.2 – Comparaison des solutions existantes

Solution	**Points forts**	**Limites**	**Adapté ?**
SAP Ariba	Complète, intégration ERP	Coût élevé (licences), délai de déploiement long, sur-mesure impossible	Non
Proactis	Gestion marchés publics	Coût prohibitif pour une PME, rigidité du workflow	Non
OpenProposal / open source	Gratuit	Documentation insuffisante, peu maintenu, pas de support	Non
Excel + partage réseau	Gratuit, familier	Non sécurisé, non traçable, fragile, pas de portail externe	Non
Développement sur mesure	**Adapté exactement aux besoins**	Délai de développement	**Oui**

1.4.2 Critique de l'existant

Aucune des solutions commerciales n'est adaptée au contexte de ce stage pour trois raisons cumulatives. D'abord, le **budget nul** interdit les licences SaaS. Ensuite, le **délai d'un mois** exclut tout déploiement et personnalisation de produit tiers. Enfin, le besoin d'un **chiffrement sélectif** des documents financiers et d'un portail public isolé dépasse les fonctionnalités standard des outils génériques.

La solution Excel actuelle, bien que pratique, expose l'entreprise à des risques réels : perte de données, accès non contrôlé aux offres financières, impossibilité de retrouver l'historique des modifications.

1.4.3 Justification du développement sur mesure

Le développement d'une application web sur mesure est la seule option répondant simultanément à toutes les contraintes :

- Workflow exact du processus N+ONE (statuts, rôles, notifications) ;
- Chiffrement AES-256-GCM intégré nativement ;
- Portail public isolé pour les sociétés externes ;
- Coût nul en licences (stack open source) ;
- Déployable sur l'infrastructure existante de N+ONE.

1.4.3.0.1 Estimation des coûts d'infrastructure En cas de déploiement en production, le coût se limiterait à l'infrastructure serveur : environ 50 à 100 €/mois sur un serveur cloud (ou nul si déployé sur les serveurs internes N+ONE), contre plusieurs milliers d'euros annuels pour une licence SAP Ariba.

Chapitre 2

Analyse, Conception et Architecture

2.1 Choix technologiques

Le choix de la stack technique a été guidé par trois critères : la rapidité de développement (délai d'un mois), la maturité des outils (documentation, communauté) et la sécurité intégrée (pas d'add-ons nécessaires pour les fonctionnalités critiques).

Table 2.1 – Comparaison et justification des choix technologiques

Couche	**Choix retenu**	**Alternatives rejetées et raison**
Backend	Django + DRF	Node.js (trop de bibliothèques à assembler) ; Spring Boot (verbeux, délai trop long)
Frontend	React.js + Vite	Angular (courbe d'apprentissage) ; Vue.js (écosystème moins mature pour tables/PDF)
Base de données	PostgreSQL	MySQL (moins performant sur JSON, TIMESTAMPTZ, recherche plein texte)
Authentification	JWT (simplejwt)	Sessions (état serveur difficile à scaler) ; OAuth (surcharge pour usage interne)
Conteneurisation	Docker Compose	Kubernetes (trop complexe pour un développeur solo, délai 1 mois)
Proxy	Nginx + HTTPS	Caddy (moins documenté) ; Gunicorn direct (pas de TLS natif)
CI/CD	GitHub Actions	Jenkins (installation, maintenance, scope creep)

2.1.0.0.1 Django a été préféré pour son administration intégrée (économie de 3 à 4 jours), son ORM prévenant nativement les injections SQL, et DRF qui fournit sérialisation, permissions et pagination sans code supplémentaire.

2.1.0.0.2 PostgreSQL offre un support natif des colonnes JSON (utile pour `AuditLog.changes`), un type `TIMESTAMPTZ` précis pour le suivi des délais, et de meilleures performances sur la recherche textuelle avec l'opérateur `ILIKE`.

2.1.0.0.3 Docker Compose permet de démarrer l'ensemble du système (frontend, backend, base de données, proxy, scheduler) en une seule commande, garantissant la reproductibilité entre environnements de développement et de démonstration.

2.2 Planification

Le projet a été découpé en quatre sprints hebdomadaires, chacun centré sur un objectif livrable.

Semaine 1 (6–12 août) Fondations : Docker Compose, Django, PostgreSQL, authentification JWT, modèle User avec rôles. Livrable : connexion fonctionnelle, endpoints auth testés via Postman.

Semaine 2 (13–19 août) Fonctionnalités métier : offres, candidats, soumissions, frontend React connecté à l'API, upload de documents. Livrable : un manager peut créer une offre, enregistrer des soumissions.

Semaine 3 (20–26 août) Sécurité et notifications : RBAC, journal d'audit, monitoring des délais, chiffrement AES-256-GCM, notifications en application. Livrable : rôles appliqués, documents sensibles chiffrés, alertes actives.

Semaine 4 (27 août – 6 septembre) Portail public, rapports PDF, tests, pipeline CI, documentation. Livrable : application complète, testée, déployable en une commande.

2.3 Modélisation des besoins

2.3.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation (figure **??**) présente les interactions entre les cinq acteurs du système et les fonctionnalités exposées. Les acteurs internes (Admin, Manager, Analyst, Viewer) s'authentifient via JWT. L'acteur externe (Société soumissionnaire) accède uniquement au portail public sans authentification.

2.3.2 Diagramme d'activité – Cycle de vie d'une offre

Le cycle de vie d'une offre est contrôlé côté serveur par la méthode `can_transition_to()` du modèle `OfferCall`. Les transitions autorisées sont :

$$\text{DRAFT} \rightarrow \text{PUBLISHED} \rightarrow \text{EVALUATION} \rightarrow \text{AWARDED}$$

À tout moment avant attribution, une offre peut être annulée (`CANCELLED`) via l'endpoint `POST /api/offers/{id}/cancel/`. Toute tentative de transition non autorisée retourne une erreur HTTP 400.

2.4 Architecture de la solution

2.4.1 Vue globale

L'application repose sur une architecture **monolithe modulaire** : un seul processus Django contenant huit applications indépendantes, déployé derrière un proxy Nginx gérant le TLS.

Le flux d'une requête authentifiée :

1. Le navigateur envoie la requête HTTPS sur le port 443.

2. Nginx termine le TLS et route vers le frontend React (assets statiques) ou vers le backend Django (`/api/*`).
3. Django traite la requête : authentification JWT, vérification du rôle, logique métier, accès PostgreSQL via l'ORM.
4. La réponse remonte par Nginx vers le client.

2.4.2 Monolithe modulaire vs microservices

Une architecture microservices aurait nécessité : API gateway, découverte de services, authentification inter-services, traçage distribué – impossible à développer seul en un mois. Le monolithe modulaire offre la séparation des responsabilités via les frontières des applications Django (chaque app possède ses propres `models.py`, `views.py`, `serializers.py`, `urls.py`), tout en conservant la simplicité opérationnelle (`docker-compose up`).

2.4.3 Services Docker Compose

Table 2.2 – Services Docker Compose

Service	Port	Rôle
`db`	5432	PostgreSQL, données persistées sur volume nommé
`backend`	8000	Django + Gunicorn, migrations automatiques au démarrage
`frontend`	3000	React SPA (build statique ou serveur de dev)
`nginx`	80, 443	Proxy inverse, terminaison TLS, redirection HTTP→HTTPS
`scheduler`	–	Boucle `check_deadlines && sleep 3600`

Tous les services disposent de vérifications de santé (*health checks*). Les services `backend` et `nginx` démarrent uniquement après que `db` soit déclaré sain (`condition: service_healthy`).

2.5 Conception détaillée

2.5.1 Modèle de données

Le modèle de données comprend sept entités principales, représentées dans le diagramme entité-relation (figure **??**).

Les choix de conception notables sont :

UUID comme clé primaire Tous les modèles utilisent `UUIDField` comme clé primaire. Contrairement aux entiers auto-incrémentés, les UUIDs ne révèlent pas le volume de données et empêchent l'énumération (`/offers/1`, `/offers/2`).

`uploaded_by` nullable Le champ `Document.uploaded_by` est nullable avec `on_delete=SET_NULL`. Les documents déposés via le portail public (sans compte utilisateur) ont ce champ à `NULL`. Les documents uploadés par le personnel ont leur auteur référencé.

Deadlines séparées `OfferCall` possède deux champs distincts : `submission_deadline` (date limite de dépôt pour les candidats) et `evaluation_deadline` (date limite d'évaluation pour les équipes internes). Le scheduler surveille les deux indépendamment.

`AuditLog.changes` en JSON Le champ `changes` stocke un objet JSON arbitraire représentant les modifications apportées à la ressource. Cette approche évite une table de colonnes rigides et s'adapte à n'importe quelle ressource auditée.

2.5.2 Conception de l'API REST

L'API respecte les conventions REST : ressources nommées, méthodes HTTP sémantiques, codes de retour standard. La pagination est systématique sur les listes (`count, next, previous, results`).

TABLE 2.3 – Principaux endpoints de l'API

Endpoint	Méthode	Description
`/api/auth/login/`	POST	Connexion, retourne access token + cookie httpOnly
`/api/auth/refresh/`	POST	Rafraîchit l'access token via le cookie
`/api/auth/logout/`	POST	Blackliste le refresh token, supprime le cookie
`/api/offers/`	GET, POST	Liste paginée, création d'offre
`/api/offers/{id}/`	GET, PATCH	Détail, modification
`/api/offers/{id}/publish/`	POST	DRAFT → PUBLISHED
`/api/offers/{id}/close/`	POST	PUBLISHED → EVALUATION
`/api/offers/{id}/award/`	POST	EVALUATION → AWARDED
`/api/offers/{id}/cancel/`	POST	Annulation (tout statut non terminal)
`/api/documents/upload/`	POST	Upload multipart, chiffrement optionnel
`/api/documents/{id}/download/`	GET	Téléchargement (déchiffrement transparent)
`/api/notifications/unread-count/`	GET	Nombre de notifications non lues
`/api/reports/offers/{id}/export/`	GET	Export PDF d'une offre
`/api/audit/logs/`	GET	Journal d'audit (ADMIN uniquement)
`/api/portal/offers/`	GET	Offres publiées (AllowAny)
`/api/portal/offers/{id}/submit/`	POST	Soumission publique (AllowAny, rate limited)

Les transitions d'état sont des endpoints POST dédiés (et non une modification PATCH du champ `status`). Cela garantit que la logique de validation de la transition est toujours exécutée côté serveur, quelle que soit la requête entrante.

2.5.3 Architecture de sécurité

2.5.3.1 Authentification JWT

Le schéma d'authentification repose sur deux tokens aux durées de vie différentes : l'access token (15 minutes) et le refresh token (7 jours).

L'access token est stocké en **mémoire React** (variable d'état dans `AuthContext`). Ce choix est délibéré : stocker le token dans `localStorage` l'exposerait aux attaques XSS, une vulnérabilité classique du top 10 OWASP. Un script malveillant injecté dans la page pourrait lire `localStorage` mais ne peut pas accéder à une variable JavaScript en mémoire appartenant à un autre contexte.

Le refresh token est stocké dans un cookie avec les attributs `HttpOnly`, `Secure`, `SameSite=Lax` et `path=/api/auth/`. L'attribut `HttpOnly` empêche tout accès JavaScript au cookie. L'attribut `path=/api/auth/` restreint l'envoi du cookie aux seules URLs `/api/auth/refresh/` et `/api/auth/logout/`, évitant qu'il soit transmis sur les requêtes du portail public.

Au chargement de l'application, un appel silencieux à `/api/auth/refresh/` restaure l'access token depuis le cookie, permettant à l'utilisateur de rester connecté sans ressaisir ses identifiants.

2.5.3.2 Contrôle d'accès basé sur les rôles (RBAC)

Chaque endpoint Django est protégé par une classe de permission DRF :

TABLE 2.4 – Matrice de contrôle d'accès

Action	ADMIN	MANAGER	ANALYST	VIEWER
Créer une offre	✓	✓		
Modifier une offre	✓	✓		
Voir toutes les offres	✓	✓	✓	
Voir offres publiées	✓	✓	✓	✓
Scorer une soumission	✓	✓	✓	
Exporter PDF	✓	✓	✓	
Gérer les utilisateurs	✓			
Consulter l'audit log	✓			

2.5.3.3 Chiffrement AES-256-GCM

Le chiffrement est appliqué **sélectivement** aux documents marqués sensibles (propositions financières, contrats). Le chiffrement de tous les fichiers ajouterait une latence inutile sur les pièces jointes anodines.

L'algorithme AES-256-GCM (mode Galois/Counter Mode) garantit simultanément :

- **La confidentialité** : les données en clair sont inaccessibles sans la clé de 256 bits.
- **L'intégrité** : le tag d'authentification de 128 bits détecte toute modification du fichier chiffré.

Pour chaque fichier chiffré, un nonce de 12 octets est généré aléatoirement. Le fichier stocké sur disque contient la concaténation : `nonce (12B) || ciphertext || tag (16B)`. La clé de chiffrement (32 octets = 256 bits) est fournie via la variable d'environnement `DOCUMENT_ENCRYPTION_KEY` et n'apparaît jamais dans le code source ni dans le dépôt Git.

2.5.3.4 Diagramme de séquence – Soumission portail public

La soumission via le portail public suit un flux en sept étapes : vérification de l'offre (statut PUBLISHED), contrôle de la deadline, validation des données, création ou récupération du candidat (upsert par email), vérification d'unicité, création de la soumission, upload du document, et notification du responsable assigné.

Ce flux est atomique : en cas d'erreur à l'étape 7 (validation du fichier), la soumission créée est supprimée pour maintenir la cohérence de la base de données.

Chapitre 3

Réalisation et Mise en Œuvre

3.1 Environnement de développement

3.1.1 Outils utilisés

Table 3.1 – Outils de développement

Outil	Version	Usage
Python	3.11	Langage backend
Django	4.2	Framework web backend
Django REST Framework	3.15	API REST
React.js	18	Framework frontend
Vite	5	Bundler frontend
TailwindCSS	3	Styles utilitaires
PostgreSQL	15	Base de données
Docker & Compose	24	Conteneurisation
Nginx	1.25	Proxy inverse, TLS
GitHub Actions	–	Pipeline CI (lint + tests)
VS Code	–	Éditeur de code
Postman	–	Tests manuels API

3.1.2 Structure du projet

Le projet est organisé en deux répertoires principaux (`backend/` et `frontend/`), complétés par la configuration Docker et Nginx. Le backend Django suit une architecture modulaire : chaque domaine fonctionnel est une application Django distincte avec ses propres modèles, vues, sérialiseurs et URLs.

```
appels-offres/
|-- docker-compose.yml
|-- nginx/
|   |-- nginx.conf
|   `-- certs/          # Certificat SSL auto-signe (demo)
|-- backend/
```

```
|   |-- config/        # Settings, urls.py racine, wsgi.py
|   `-- apps/
|       |-- authentication/   # User, JWT, RBAC, seed_demo
|       |-- offers/           # OfferCall, state machine
|       |-- candidates/       # Candidate, Submission
|       |-- documents/        # Upload, AES-256-GCM
|       |-- notifications/    # Notification, check_deadlines
|       |-- reports/          # Export PDF (reportlab)
|       |-- audit/            # AuditLog, signaux Django
|       `-- portal/           # Endpoints publics (AllowAny)
|-- frontend/
|   `-- src/
|       |-- contexts/         # AuthContext
|       |-- pages/            # Login, Dashboard, offers, portal...
|       |-- services/         # Wrappers API par domaine
|       `-- hooks/            # useAuth, useNotifications
`-- .github/workflows/ci.yml  # GitHub Actions
```

Listing 3.1 – Structure du projet

3.1.3 Pipeline d'intégration continue

À chaque push sur la branche `main`, GitHub Actions exécute automatiquement :

1. **Lint** : `flake8` vérifie le style et la qualité du code Python (longueur de lignes, imports inutilisés, variables non utilisées).
2. **Tests** : `pytest-django` exécute les tests d'intégration avec une base de données PostgreSQL dédiée au pipeline.

3.2 Implémentation du backend

3.2.1 Authentification et RBAC

Le modèle `User` étend `AbstractBaseUser` de Django pour définir le champ `email` comme identifiant principal (et non le nom d'utilisateur). Un champ `role` de type `CharField` avec des choix énumérés (`ADMIN`, `MANAGER`, `ANALYST`, `VIEWER`) définit le rôle de chaque utilisateur.

```python
class User(AbstractBaseUser, PermissionsMixin):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4)
    email = models.EmailField(unique=True)
    full_name = models.CharField(max_length=150)
    role = models.CharField(max_length=10, choices=ROLE_CHOICES, default='VIEWER')
    department = models.CharField(max_length=100, blank=True)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
```

Listing 3.2 – Modèle User

Les permissions DRF sont définies comme classes réutilisables : `IsManagerOrAdmin` (création/modification d'offres), `IsAnalystOrAbove` (lecture de toutes les offres, évaluation), `IsAdminRole` (gestion des utilisateurs, journal d'audit). Ces classes sont appliquées au niveau de chaque vue, centralisées dans `apps/authentication/permissions.py`.

L'endpoint de connexion génère un access token (durée 15 minutes) retourné dans le corps de la réponse JSON, et un refresh token (durée 7 jours) placé dans un cookie sécurisé. La bibliothèque `djangorestframework-simplejwt` gère la génération, la validation et la révocation des tokens.

3.2.2 Gestion des offres et machine à états

Le modèle `OfferCall` implémente une machine à états explicite via la méthode `can_transition_to()` qui vérifie si la transition demandée est autorisée depuis le statut courant.

```
ALLOWED_TRANSITIONS = {
    'DRAFT':      ['PUBLISHED', 'CANCELLED'],
    'PUBLISHED':  ['EVALUATION', 'CANCELLED'],
    'EVALUATION': ['AWARDED', 'CANCELLED'],
    'AWARDED':    [],
    'CANCELLED':  [],
}

def can_transition_to(self, new_status):
    return new_status in ALLOWED_TRANSITIONS.get(self.status, [])
```

Listing 3.3 – Machine à états des offres

Chaque transition est un endpoint POST dédié (`/publish/`, `/close/`, `/award/`, `/cancel/`). Cette conception interdit toute modification directe du champ `status` via PATCH, garantissant que la logique de validation est toujours exécutée.

3.2.3 Documents et chiffrement AES-256-GCM

L'upload de documents s'effectue en multipart/form-data. Avant le stockage, le fichier passe par une validation (type MIME dans la liste blanche PDF/DOCX/XLSX, taille inférieure à 10 Mo), puis par le module de chiffrement si le flag `is_sensitive` est activé.

```
def encrypt_file(data: bytes, key: bytes) -> bytes:
    nonce = os.urandom(12)
    aesgcm = AESGCM(key)
    ciphertext = aesgcm.encrypt(nonce, data, None)
    return nonce + ciphertext  # nonce (12B) || ciphertext || tag (16B)
```

Listing 3.4 – Chiffrement AES-256-GCM

Le déchiffrement est **transparent** pour le client : lors du téléchargement, Django détecte `is_encrypted=True`, lit les 12 premiers octets comme nonce, déchiffre le reste, et retourne le fichier en clair dans la réponse HTTP. La clé ne transite jamais en dehors du processus Django.

3.2.4 Portail public

L'application `portal` est isolée : elle ne partage aucune classe de permission ni aucun sérialiseur avec les applications internes. Elle importe uniquement les modèles (`OfferCall`, `Candidate`, `Submission`, `Document`).

La vue `PublicSubmitView` implémente trois protections :

1. **Rate limiting** : décorateur `@ratelimit(key='ip', rate='5/h', block=True)` de la bibliothèque `django-ratelimit`. Un script automatisé ne peut pas inonder la base de données.
2. **Vérification de deadline** : `timezone.now() > offer.submission_deadline` retourne HTTP 400 si la date limite est dépassée.
3. **Unicité** : une société ne peut pas soumettre deux fois pour la même offre (identifiée par email).

3.2.5 Audit et notifications

Le journal d'audit est alimenté par des appels explicites à `AuditLog.objects.create()` dans chaque vue lors des opérations d'écriture. Chaque entrée enregistre : l'utilisateur (nullable pour le portail), l'action, le type de ressource, son identifiant, les modifications en JSON et l'adresse IP.

Le scheduler s'exécute dans un conteneur Docker dédié en boucle :

```
while true; do
    python manage.py check_deadlines && sleep 3600 || sleep 60;
done
```

Listing 3.5 – Boucle du scheduler

La commande `check_deadlines` interroge la base de données pour toutes les offres dont la deadline approche (dans les 24 heures) et crée les notifications correspondantes pour les responsables assignés.

3.3 Implémentation du frontend

3.3.1 Gestion de l'authentification

L'`AuthContext` React centralise l'état d'authentification : l'access token est stocké dans une variable d'état (mémoire), jamais dans `localStorage` ou `sessionStorage`. Au montage de l'application, un appel silencieux à `/api/auth/refresh/` restaure le token depuis le cookie httpOnly si la session est encore valide.

Un intercepteur Axios ajoute automatiquement le header `Authorization: Bearer <token>` à chaque requête, et gère le rafraîchissement transparent en cas de réponse 401 (token expiré).

3.3.2 Routing protégé par rôle

Les routes React sont enveloppées dans des composants de garde : `PrivateRoute` (authentification requise) et `RoleGuard` (rôle minimum requis). Un utilisateur VIEWER tentant d'accéder à la page de création d'offre est redirigé vers le dashboard.

3.3.3 Portail public

Le portail utilise une instance Axios séparée (`portalApi.js`) sans header `Authorization`. Les pages `/portal` et `/portal/:id` sont accessibles sans connexion et ne partagent aucun état avec l'application interne.

3.4 Interfaces et fonctionnalités

3.4.1 Tableau de bord

3.4.2 Liste et filtrage des offres

La page de liste affiche les offres sous forme de tableau paginé avec filtres par statut, catégorie et recherche textuelle. Les badges colorés signalent l'état de chaque offre (vert : publiée, orange : en évaluation, rouge : annulée).

3.4.3 Détail d'une offre

La page de détail est organisée en onglets : informations générales, soumissions des candidats (avec scores), et documents associés (avec indicateur de chiffrement AES-256).

3.4.4 Formulaire de création d'offre

3.4.5 Évaluation d'une soumission

3.4.6 Portail public

3.4.7 Journal d'audit

3.4.8 Rapport PDF

3.5 Tests et validation

3.5.1 Tests automatisés

Les tests d'intégration couvrent les scénarios critiques du système :

Table 3.2 – Couverture des tests automatisés

Module	Cas testés
Authentification	Connexion valide, credentials incorrects, refresh token, déconnexion
RBAC	Accès autorisé/refusé par rôle sur les endpoints critiques
Offres	Création, transitions d'état autorisées et refusées, filtres
Portail	Soumission valide, deadline dépassée, doublon, fichier invalide
Documents	Upload, download, détection chiffrement

Les tests utilisent des fixtures par rôle (`admin_user`, `manager_user`, `analyst_user`, `viewer_user`) et une base de données PostgreSQL dédiée. Chaque test est isolé dans une transaction annulée après exécution.

3.5.2 Résultats CI

Le pipeline GitHub Actions exécute à chaque push sur `main` : **12 tests pytest** couvrant les cas critiques, et le lint **flake8** sur l'ensemble du code Python. Les deux étapes passent au vert.

3.5.3 Tests manuels

Un script `seed_demo.py` peuple la base de données avec un scénario représentatif : 6 utilisateurs (un par rôle), 6 offres à différents stades du cycle de vie, 6 candidats et 8 soumissions avec scores. Ce scénario a permis de valider le parcours complet : création d'offre → publication → soumission portail → évaluation → attribution → export PDF → consultation journal d'audit.

Conclusion

Récapitulatif des réalisations

Au terme de ce stage d'un mois, l'application web de gestion des appels d'offres a été développée, testée et déployée dans son intégralité. Les objectifs fixés en début de stage ont été atteints :

- Authentification sécurisée JWT avec cookie httpOnly scopé, contrôle d'accès par rôles (RBAC) sur l'ensemble des endpoints ;
- Cycle de vie complet des appels d'offres (DRAFT → PUBLISHED → EVALUATION → AWARDED) avec machine à états côté serveur ;
- Portail public permettant aux sociétés externes de déposer leurs propositions sans compte, avec rate limiting et vérification des deadlines ;
- Chiffrement AES-256-GCM sélectif des documents sensibles, déchiffrement transparent au téléchargement ;
- Notifications automatiques par le scheduler Docker ;
- Export PDF des synthèses d'offres ;
- Journal d'audit complet sur toutes les opérations d'écriture ;
- Pipeline CI GitHub Actions (lint + tests) actif sur chaque push.

Difficultés rencontrées

3.5.3.0.1 Isolation du portail public La principale difficulté architecturale a été de concevoir le portail public comme une surface réellement isolée de l'API interne. Le problème du cookie de rafraîchissement – qui, sans restriction de chemin, aurait été envoyé sur toutes les requêtes du portail – a nécessité de restreindre le cookie à `path=/api/auth/`, une subtilité non documentée dans la plupart des tutoriels JWT.

3.5.3.0.2 Chiffrement sélectif L'intégration du chiffrement AES-256-GCM a soulevé la question de la gestion des clés : stocker la clé dans le code source ou dans `docker-compose.yml` aurait constitué une faille évidente. La solution par variable d'environnement (`DOCUMENT_ENCRYPTION_KEY`) et l'exclusion des fichiers `.env` du dépôt Git ont résolu ce problème.

3.5.3.0.3 Pipeline CI Les premiers commits ont déclenché des échecs de pipeline liés à des migrations manquantes et des violations de lint. La correction a nécessité de comprendre le flux de migration Django dans un environnement de test isolé.

Compétences acquises

Ce stage a été l'occasion de mettre en pratique et d'approfondir plusieurs compétences :

- Développement d'API REST sécurisées avec Django REST Framework ;
- Sécurité web applicative (OWASP Top 10, JWT, RBAC, chiffrement) ;
- Conteneurisation avec Docker Compose et architecture multi-services ;
- Intégration continue avec GitHub Actions ;
- Conception de systèmes : modèle de données, machine à états, audit.

Perspectives

L'application est fonctionnelle et déployable, mais plusieurs améliorations pourraient être envisagées pour une mise en production réelle :

Certificat SSL valide Remplacer le certificat auto-signé par un certificat Let's Encrypt pour éliminer les avertissements navigateur.

Notifications par email Ajouter l'envoi d'emails (SMTP) en complément des notifications en application.

Rotation des clés AES Implémenter un mécanisme de re-chiffrement des documents lors d'un changement de clé.

Optimistic locking Prévenir les conflits d'édition simultanée sur les offres via un champ de version.

Docker Secrets Remplacer les variables d'environnement par Docker Secrets ou un gestionnaire de secrets dédié (Vault) en production.

Chapitre 4

docker-compose.yml

```yaml
version: '3.9'

services:
  db:
    image: postgres:15
    environment:
      POSTGRES_DB: ${DB_NAME:-offres_db}
      POSTGRES_USER: ${DB_USER:-postgres}
      POSTGRES_PASSWORD: ${DB_PASSWORD:-postgres}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER:-postgres}"]
      interval: 10s
      timeout: 5s
      retries: 5

  backend:
    build: ./backend
    command: >
      sh -c "python manage.py migrate --noinput &&
             python manage.py collectstatic --noinput &&
             python manage.py runserver 0.0.0.0:8000"
    environment:
      SECRET_KEY: ${SECRET_KEY}
      DOCUMENT_ENCRYPTION_KEY: ${DOCUMENT_ENCRYPTION_KEY}
    depends_on:
      db:
        condition: service_healthy

  scheduler:
    build: ./backend
    command: >
      sh -c "while true; do
               python manage.py check_deadlines && sleep 3600 || sleep 60;
             done"
    depends_on:
      db:
        condition: service_healthy

  frontend:
    build: ./frontend
    ports:
      - "3000:3000"

  nginx:
    build: ./nginx
    ports:
      - "80:80"
```

```
      - "443:443"
    depends_on:
      - backend
      - frontend

volumes:
  postgres_data:
```

Listing 4.1 – docker-compose.yml complet

Chapitre 5

Exemple d'entrée AuditLog

Exemple d'enregistrement dans le journal d'audit pour une opération de publication d'une offre :

```
{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "user": {
    "id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
    "email": "manager@nonedc.ma",
    "full_name": "Karim Benchekroun",
    "role": "MANAGER"
  },
  "action": "UPDATE",
  "resource_type": "OfferCall",
  "resource_id": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "changes": {
    "status": "PUBLISHED",
    "published_at": "2025-08-15T09:32:14.000Z"
  },
  "ip_address": "192.168.1.42",
  "timestamp": "2025-08-15T09:32:14.123Z"
}
```

Listing 5.1 – Entrée AuditLog – publication d'une offre

Chapitre 6

Manuel utilisateur

Connexion

Accéder à l'application via le navigateur. Saisir l'adresse email et le mot de passe. Cliquer sur *Se connecter*. En cas d'erreur, vérifier les identifiants.

Créer un appel d'offres

1. Dans le menu latéral, cliquer sur **Offres** puis **Nouvelle offre**.
2. Renseigner le titre, la description, la catégorie, le budget estimatif et les dates limites.
3. Cliquer sur **Enregistrer**. L'offre est créée en statut BROUILLON.
4. Pour la publier, ouvrir le détail de l'offre et cliquer sur **Publier**.

Évaluer les soumissions

1. Ouvrir le détail de l'offre, onglet **Soumissions**.
2. Cliquer sur la soumission à évaluer.
3. Saisir le score technique et le score financier.
4. Cliquer sur **Enregistrer l'évaluation**.

Portail public (sociétés externes)

1. Accéder à `https://[domaine]/portal`.
2. Consulter la liste des appels d'offres ouverts.
3. Cliquer sur une offre pour voir le descriptif complet.
4. Cliquer sur **Soumettre une offre** et remplir le formulaire (coordonnées, numéro d'enregistrement, fichier PDF de la proposition).
5. Conserver le numéro de soumission retourné à titre de référence.

Chapitre 7

Résultats du pipeline CI

Le pipeline s'exécute en deux étapes séquentielles :

1. `flake8 .` – vérification du style Python (0 erreur)
2. `pytest -ds=config.settings` – 12 tests, 0 échec, 0 saut